

1. E-commerce Platform Search Function

```
using System;

class Product
{
    public int Id;
    public string Name;

    public Product(int id, string name)
    {
        Id = id;
        Name = name;
    }
}

class Program
{
    static void Main()
    {
        Product[] products = {
            new Product(101,"Laptop"),
            new Product(102,"Mobile"),
            new Product(103,"Shoes"),
            new Product(104,"Watch")
        };

        int key = 103;

        // Linear Search
        foreach (Product p in products)
            if (p.Id == key)
                Console.WriteLine("Linear Search: " + p.Name);

        // Binary Search
        int l = 0, r = products.Length - 1;
        while (l <= r)
        {
            int m = (l + r) / 2;

            if (products[m].Id == key)
            {
                Console.WriteLine("Binary Search: " + products[m].Name);
            }
        }
    }
}
```

```

        break;
    }
    else if (products[m].Id < key)
        l = m + 1;
    else
        r = m - 1;
    }
}
}
}

```

Output

Linear Search: Shoes

Binary Search: Shoes

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

Binary Search is more suitable because it significantly reduces the number of comparisons and has better Time Complexity.

2. Financial Forecasting

using System;

```

class Forecast
{
    static double Predict(double amount, double rate, int years)
    {
        if (years == 0)
            return amount;

        return Predict(amount * (1 + rate / 100), rate, years - 1);
    }

    static void Main()
    {
        double result = Predict(1000, 10, 5);
        Console.WriteLine("Future Value = " + result);
    }
}

```

Output

Future Value = 1610.51

Time Complexity: **$O(n)$** where n is the number of years.

Optimization:

- Use an iterative loop instead of recursion to reduce stack usage.